Comprendre les évolutions du projet LoRa APRS

Référence iGate : branche `rxt`, commit `2cebfe6` du 13 octobre 2026. Lecture et présentation mises à jour le 15 octobre 2026.

Les explications suivantes accompagnent les diapositives. Le pic de taux observé par Jon est reporté à une autre tâche.

1. Comprendre le projet LoRa APRS

Cette présentation explique le fonctionnement du fork iGate et son articulation avec Graywolf. Elle couvre les évolutions globales par rapport à la base CA2RXU, puis les modifications RXT et natives jusqu'au commit 2cebfe6. Les appareils déjà déployés ne sont pas supposés exécuter ce même état. La question du pic de RX_rate, RelRate et DrpRate est reportée à une autre tâche.

2. La base CA2RXU et les évolutions

La base CA2RXU fournit déjà les fonctions LoRa, iGate APRS-IS, digipeater, interfaces TNC et WebUI. Il ne faut pas attribuer leur existence au fork. L'historique du fork montre notamment le travail sur RXT, le flux de réception JSON, les taux de télémétrie, les alias régionaux, le chemin retour appris, les réponses locales, le contrôle distant authentifié et le transport radio natif. Ces ajouts prolongent plusieurs circuits existants plutôt qu'ils ne remplacent tout le firmware. Lire git log et les modules concernés pour attribuer une correction précise.

3. Les rôles dans le réseau

Un tracker crée le contenu APRS. Un digipeater réémet une réception après application des règles de chemin. Un iGate relie la radio et APRS-IS. Graywolf est l'application qui conserve et présente les réceptions, les positions et les mesures. Une carte peut cumuler plusieurs de ces fonctions selon sa configuration. RXT ne rend pas nécessaire une connexion Internet sur chaque relais : les mesures voyagent sur la radio jusqu'au récepteur qui les exporte.

4. Le paquet TNC2

Dans SOURCE>DEST,CHEMIN:INFORMATION, le signe > termine la source et le premier deux-points termine l'en-tête. Le début du champ d'information indique généralement le type APRS, par exemple > pour un statut. L'en-tête TNC2 est textuel, mais le champ d'information peut contenir des octets binaires, notamment Mic-E. Une ligne affichable ne constitue donc pas toujours une représentation fidèle de tous les octets. Le flux JSON utilise Base64 comme copie faisant foi.

5. Les identités étendues

Le modèle textuel peut conserver un suffixe opaque après le tiret, par exemple -GS, ou un nombre supérieur à 15. Le fait de conserver une réception ne signifie pas que tous les protocoles et toutes les fonctions accepteront ensuite cette identité. Graywolf vérifie la représentation exacte lorsqu'il doit sortir vers AX.25. Dans l'iGate, encodeAddressAX25 de src/kiss_utils.cpp utilise toInt et ne fournit pas cette même validation stricte. Éviter d'utiliser sa sortie KISS pour garantir la conservation d'une identité étendue. Les règles d'admission APRS-IS et de relais restent elles aussi distinctes de la conservation JSON.

6. TNC2 et la frontière AX.25

Le TNC2 direct transporte des enregistrements texte sur un port série ou une connexion TCP. KISS transporte des trames AX.25 binaires. Choisir TNC2 évite une conversion AX.25 quand la radio et l'application savent déjà échanger du texte. Dans l'iGate, tnc.kissProtocol choisit le mode pour la série et le serveur TCP. Le port du serveur TNC est défini à 8001 dans src/tnc_utils.cpp. Dans Graywolf, les identités AX.25 incompatibles sont refusées à la conversion. Ne pas étendre cette garantie à l'encodeur KISS historique de l'iGate.

7. Le parcours dans le code iGate

Dans la boucle de LoRa_APRS_iGate.cpp, receivePacket lit la radio et produit le paquet ainsi qu'une copie destinée au producteur JSON. Le contexte local est capturé avant qu'une autre opération radio ne change les mesures. Le paquet accepté passe ensuite aux fonctions APRS-IS et digi si elles sont actives. Les sauts RXT sont décodés pour JSON, le tableau Web et les sorties TNC. MQTT et le journal SD suivent leurs propres conditions. Les charges utiles reçues avec CRC valide mais refusées par la syntaxe ou la politique applicative peuvent encore être conservées par le circuit JSON en fonctionnement normal, hors ecoMode.

8. RXT décrit une réception distante

Dans cet exemple de septembre, les mesures portées par SOMTNX concernent sa réception de CAVECR. Elles ne décrivent pas la réception de SOMTNX par TSRXBX. Cette dernière est la mesure locale du récepteur final. En combinant le chemin, les tuples des relais et la mesure locale, l'application peut décrire plusieurs liaisons dirigées. Une liaison sans mesure ne doit pas recevoir une qualité inventée. Les noms de cet exemple ne constituent pas une déclaration de l'inventaire actuel des sites.

9. Les quatre mesures RXT

RSSI donne un niveau de signal et SNR le rapport signal sur bruit mesuré par le récepteur. FO est l'écart de fréquence observé, sans permettre à lui seul d'attribuer l'erreur à un seul oscillateur. TTH couvre la fin de réception jusqu'à la fin de l'émission relayée. Dans le code il inclut le séjour en file, l'attente de canal CAD/DIFS/backoff et le temps d'antenne calculé pour la trame transmise. Ce délai ne mesure donc pas le seul temps de traitement du processeur. Les valeurs RXT sont quantifiées : SNR de -24 à +20 dB par pas de 0,5 dB dans le code actuel, RSSI de -130 à -41 dBm, FO non linéaire et TTH exponentiel selon le profil LoRa.

10. RXT v2 : identifier le relais

Le tuple texte v2 ajoute un caractère I devant les quatre mesures. Ce caractère est une empreinte de l'indicatif effectivement écrit dans le chemin, en majuscules, suffixe compris et sans étoile : FNV-1a 32 bits, modulo 89, plus 33. Il n'est donc ni un identifiant global unique ni une table centralisée. Le décodeur confronte les empreintes aux relais utilisés et à leur ordre. Une collision peut être résolue par l'ordre si assez de tuples l'établissent. S'il manque des tuples et que plusieurs propriétaires restent possibles, identity_status vaut unresolved. Un chemin ne permet pas de retrouver un indicatif qu'aucun relais n'a inscrit. Le firmware conserve au plus trois tuples et n'ajoute pas de RXT aux messages APRS, ACK ou REJ, même encapsulés.

11. Le bloc RXT caché en mode texte

Les octets 3C FF 01 annoncent le transport LoRa texte. Le paquet TNC2 est suivi, s'il comporte un RXT reconnu, de 00 puis du marqueur 01 et des caractères des tuples, sans accolades. toExtensionPayload effectue cette séparation. fromExtensionPayload contrôle l'extension contre le chemin avant de reconstruire la représentation interne en trailer. Les lecteurs de chaîne C s'arrêtent au nul et traitent seulement le paquet d'origine. Cela évite que ces iGates transmettent sur APRS-IS le RXT ajouté dans le champ information. La compatibilité dépend de ce comportement de lecture et n'est pas une garantie pour tout firmware tiers. En interne texte, la reconnaissance d'un commentaire ressemblant à un trailer conserve un risque de collision. Le RXT natif reste séparé.

12. La trame radio native

Le format natif 0 transporte le même contenu TNC2 sous forme compacte. Les adresses classiques utilisent un codage court, les formes atypiques peuvent rester en texte, les positions prises en charge utilisent un codage binaire, et le reste du texte peut utiliser Huffman si le résultat est plus court. L'encodeur vérifie le retour exact en décodant son résultat. Une disposition non reproductible avec le codage spécialisé utilise le champ information brut. Si la trame entière ne peut pas être encodée, l'émission se replie en texte. Les messages et Mic-E peuvent donc voyager dans une enveloppe native avec leur information brute, sans codage sémantique spécialisé. Le mode 2 émet deux copies et sert aux essais de coexistence. Ce format expérimental n'augmente pas à lui seul la puissance ni la sensibilité radio.

13. Conserver l'identité en natif

Le RXT natif contient un masque de positions dans le chemin et quatre octets de mesures par position. Il n'a pas besoin de l'empreinte texte pour nommer son propriétaire. Depuis le correctif 2cebfe6, le paquet TNC2 et ce contexte indexé restent séparés à la réception et dans la file. remapRxt suit les noms complets lorsque la réécriture retire un alias. Une entrée de chemin retirée perd sa mesure. Pour un indicatif répété, l'association par rang ne reste possible que si son nombre d'occurrences reste identique. Les tuples v2 dérivés pour les journaux ne remplacent pas le contexte natif. Une sortie ensuite choisie en texte retrouve néanmoins l'attribution par empreinte. Les essais du 13 octobre emploient F4MLV-18 et F6DEV-10, empreinte C commune, avec une seule mesure, puis vérifient la retransmission RF et l'attente en file.

14. Le chemin et les mesures

La présence d'un indicatif utilisé dans le chemin établit le passage déclaré par ce relais, mais ne fournit pas automatiquement ses mesures. Le décodeur signale les relais sans données par NA. La portion utilisée du chemin se termine au dernier élément marqué par *. Le code accepte aussi bien plusieurs marques que la convention où seul le dernier relais porte la marque. Les éléments restant après ce dernier * ne sont pas présentés comme des sauts déjà effectués.

15. Lire une mesure historique

Ces nombres viennent de l'exemple de Jon repris dans le PowerPoint original. Ils ne sont pas une nouvelle mesure réalisée pendant cette mise à jour. Le signe ≥ devant +12 dB reflète la plage de codage de l'époque. Le code actuel étend le SNR à +20 dB, et le même caractère ne doit pas être interprété avec une ancienne échelle. Le TTH de 594 ms est le délai de ce relais pour ce paquet, pas une moyenne du réseau.

16. RXT et télémétrie de station

RXT porte des mesures attachées à une réception particulière. La télémétrie de station agrège des tensions et des compteurs d'activité. Dans telemetry_utils.cpp les taux sont calculés à partir des compteurs et du temps réellement écoulé, en paquets par heure. RX_rate compte les réceptions acceptées après filtrage blacklist, RelRate les mises en file acceptées par le digi et DrpRate les rejets selon les règles applicatives. Ces ensembles ne constituent pas un bilan de livraison : RX_rate = RelRate + DrpRate n'est pas une identité du code. L'interprétation du pic signalé par Jon n'est pas traitée ici.

17. La télémétrie dans la WebUI

La capture de septembre illustre la présentation des canaux et des unités, et ne prouve pas à elle seule le comportement du firmware actuel. Les canaux d'activité sont publiés avec les noms RX_rate, RelRate et DrpRate et l'unité pkt/h. Le code remet les compteurs à zéro quand il génère le rapport et borne chaque valeur à 8280. Ces taux sont actuellement ajoutés avec la télémétrie de tension encodée, lorsqu'au moins un canal de tension est sélectionné et que la télémétrie météo est inactive. Le taux de relais porte sur l'acceptation en file, pas sur une confirmation radio ou une livraison au destinataire.

18. Le flux JSON de réception

Les endpoints sont /api/v1/aprs/stream pour le flux NDJSON et /api/v1/aprs/events pour la consultation des événements récents. Les événements rx contiennent packet.raw_tnc2_base64 et information.raw_base64. Les projections texte n'apparaissent que si elles sont sûres en UTF-8 et dépourvues de caractères de contrôle. Les mesures locales, paramètres radio et sauts RXT sont séparés du paquet. Le curseur combine un identifiant de démarrage et une séquence. La file conserve dix événements en RAM, et le protocole signale gap lorsque le curseur ne peut plus être satisfait, notamment après redémarrage ou perte d'historique. Ce mécanisme fournit une reprise bornée, pas un archivage illimité.

19. Ce que Graywolf exploite

Selon le code et le guide local Graywolf, chaque source JSON/RXT est consommée indépendamment. Graywolf conserve les octets puis dérive un paquet TNC2 si l'enveloppe peut être analysée. Il décode les informations APRS qu'il reconnaît et conserve aussi les réceptions non interprétables. Une liaison RXT distante provient des mesures déclarées. La liaison locale finale est calculée depuis le dernier relais utilisé, ou la source en direct, vers le récepteur. Aucun TTH local n'est inventé. Le tracé sur carte exige des positions connues. Les couleurs vert/orange expriment l'âge des observations, pas la qualité radio.

20. Le TNC2 direct dans Graywolf

La page TNC2 de Graywolf configure les connexions TCP et série et persiste ces réglages. Une connexion de réception ne donne pas l'autorisation d'émettre. Pour émettre, Graywolf exige un transport, une source exacte, un canal autorisé et une limite de taille. Une acceptation par sa file ne prouve pas l'émission RF ni la livraison. L'iGate doit également autoriser LoRa TX et utiliser le mode TNC2. Le texte entrant est trim() par l'iGate : la garantie des octets écrits par Graywolf sur le port ne prouve pas que les espaces de bord survivront au firmware.

21. Les échanges dans les deux sens

Deux chemins logiciels coopèrent. Les messages, leurs reprises et les balises peuvent sortir de Graywolf par TNC2 vers une radio. Les réceptions et leurs mesures entrent par JSON/RXT. Le flux JSON n'autorise pas à lui seul une émission. Le mot natif est ambigu : le TNC2 direct est une interface textuelle entre logiciels et TNC, tandis que la trame native LoRa est un format binaire sur la radio. L'un n'exige pas l'autre. Le firmware peut recevoir du TNC2 puis émettre une trame texte ou native selon lora.txFormat.

22. Le chemin retour des messages

L'apprentissage se fait à partir d'un paquet réellement reçu par LoRa, avant les réponses locales. La partie utilisée du chemin est inversée en excluant les alias et éléments Internet. Pour SOURCE>A,B*,WIDE2-1, l'iGate apprend B,A. La première copie retenue pendant la fenêtre locale fournit la route, ce qui favorise la première arrivée et ne prétend pas sélectionner le signal le plus fort. La route expire avec rememberStationTime et disparaît au redémarrage. Sans route valable, les messages descendants APRS-IS ne sont pas transmis, sans repli sur beacon.path. Cette règle doit être distinguée des réponses RF locales à une requête, pour lesquelles un repli beacon.path peut exister.

23. Relais et réponses maîtrisés

La déduplication locale de packet_dedup utilise par défaut 25 secondes, avec des consommations séparées pour le relais et l'apprentissage de route. Sa clé tient compte de la destination APRS dans le cas Mic-E, où elle porte des données de position. Elle ne remplace pas la déduplication APRS-IS et ne coordonne pas les iGates. Le traitement local peut renvoyer un ACK à une copie répétée afin de récupérer un ACK perdu tout en évitant une nouvelle réponse ou exécution. Les correctifs de septembre conservent le reply-ack et font répondre l'identité réellement adressée. Les alias régionaux sont configurables et bornés, et le code empêche un retour par son propre relais. Les limites de taille se vérifient avant émission : absence de place pour le RXT signifie relais sans nouveau tuple, et paquet APRS trop long signifie refus, sans troncature.

24. Les commandes radio authentifiées

Le contrôle distant est distinct du trafic APRS ordinaire et de l'authentification d'une connexion APRS-IS. remote_auth vérifie l'identité du contrôleur configuré et une étiquette HMAC-SHA256 calculée sur un message qui lie notamment contrôleur, cible, compteur et commande. Une fenêtre de compteurs persistée sert à rejeter les commandes déjà consommées. Le secret doit être configuré pour ce mécanisme. La signature ne chiffre pas le contenu de la commande et n'authentifie pas tous les paquets entendus. La validation RF du 28 septembre documente une commande EM=OFF acceptée et une copie relayée ignorée sans seconde exécution.

25. Les essais documentés

Les guides Graywolf décrivent les observations Mic-E et identité étendue de septembre. La validation complémentaire du 28 septembre décrit une livraison à un T-Deck par deux relais physiques isolés par des profils radio, puis un ACK reçu directement par l'iGate. Le chemin d'apprentissage était synthétique : ce détail borne la conclusion. Les essais du 13 octobre capturent les octets RF retransmis et valident l'identité RXT en présence d'empreintes en collision, y compris avec un paquet en attente. Ces résultats ne valident ni la portée d'un site ni tous les profils et chemins possibles. Voir les comptes rendus précis cités sur la diapositive.

26. Le contexte radio de Jon

Le profil vient du récit de Jon conservé dans le PowerPoint de septembre. Il donne le contexte de l'exemple et ne renseigne pas l'état actuel de chaque équipement. Les temps d'antenne et le codage TTH dépendent des paramètres LoRa. Comparer des durées entre SF7 et SF12 sans tenir compte du profil ne permet pas d'isoler un effet du code. Les gains de compression publiés sur corpus sont des calculs sur ces corpus et ces profils, pas une mesure de la capacité ou de la portée du réseau de Jon.

27. Les limites à connaître

La réception native demande le codec correspondant. Un firmware qui ne le comprend pas peut ignorer les paquets, et certains anciens builds du fork les traitaient imparfaitement comme du texte. Le mode texte avec extension cachée préserve le paquet APRS pour les lecteurs qui s'arrêtent au nul, mais leurs relais perdent les mesures cachées. Les empreintes v2 peuvent rester ambiguës. Un format v1 résiduel n'est pas automatiquement traduit en v2. Le chemin retour apprend une route pour un iGate donné, mais ne coordonne pas plusieurs iGates qui transmettent chacun le même message descendant. Ces limites expliquent pourquoi une option implémentée et un essai concluant ne garantissent pas tous les déploiements.

28. Où lire les mécanismes

Commencer dans la boucle de LoRa_APRS_iGate.cpp permet de suivre l’ordre des appels plutôt que de lire les modules isolément. Pour une réception, suivre receivePacket, captureRxtRxContext, les consommateurs APRS-IS et digi, getDecodedRxtMetrics puis recordRx et les sorties TNC. Pour un relais, suivre generateDigipeatedPacket, addToOutputPacketBuffer, processOutputPacketBuffer et sendNewPacket. rxt_protocol.cpp gère le transport texte et l’attribution par empreinte. native_aprs.cpp gère le codec et l’association indexée. Consulter ensuite les tests hôte et les comptes rendus RF pour distinguer une garantie de fonction d’une observation matérielle.

29. Références et contributions

Les fichiers du dépôt cités sur les diapositives constituent les références de lecture. Les comptes rendus sont des observations documentées dont il faut respecter le montage et les limites. Les brouillons NOTE_TECHNIQUE_RXT et REPONSE_JON précédents ne sont pas utilisés comme preuves indépendantes. Le dépôt Graywolf associé a été lu pour expliquer ses propres garanties, distinctes de celles du firmware iGate. Aucune publication ni aucun déploiement n’accompagne ce document.